

Abstract

Convolutional Network의 구조를 end-to-end, pixels-to-pixels 방식으로 학습시켜 semantic segmentation 분야에서 SOTA 달성

임의의 사이즈의 이미지를 인풋으로 넣었을 때, 동일한 사이즈의 아웃풋이 나오도록 fully convolutional network를 구성하여 사용한다.

기존 classification network인 AlexNet, VGGnet 등을 fine tuning하여 사용하며 coarse(전반적인) 정보와 fine(세밀한) 정보를 혼합시킨다.

Introduction

Convnet의 유용성은 classification 뿐만 아니라 object detection, par and key point prediction, local correspondence로 점차 발전해왔다.

이처럼 점점 더 세부적인 정보를 예측하게 되면서 픽셀 단위의 예측까지 관심을 가지게 되었는데 이를 segmentation이라 한다.

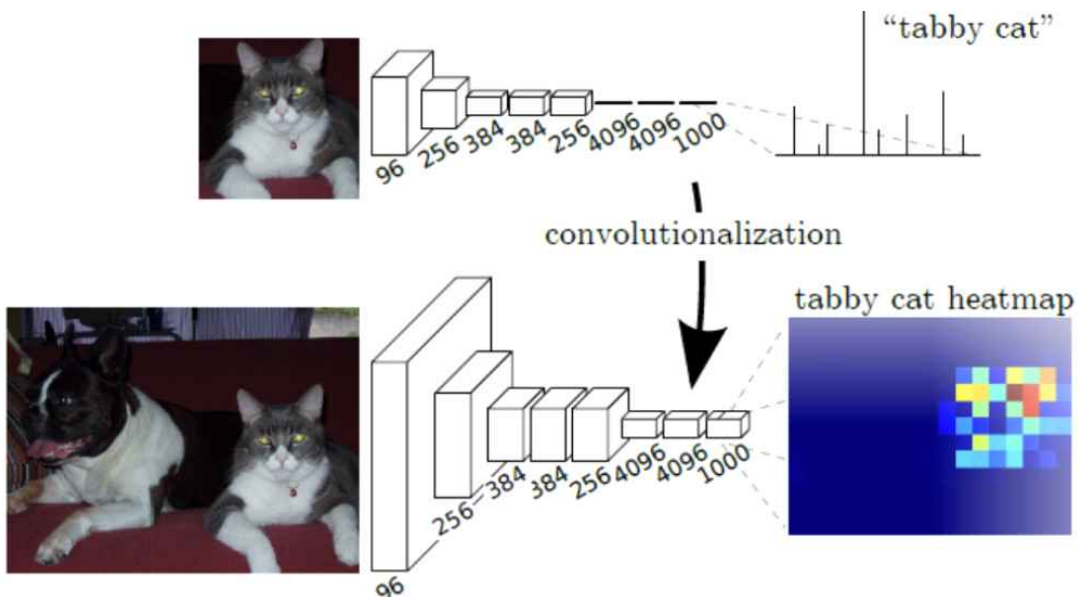
본 논문은 픽셀 단위의 예측을 위한 최초의 end-to-end 방식이자 지도 사전 학습을 활용한다.

기존의 네트워크에서는 dense한 형태의 아웃풋이 나오나 본 논문에서는 동일한 사이즈의 아웃풋을 리턴하기 위해 upsampling 기법을 활용한다. 따라서, 이미지에 대한 전처리나 후처리가 필요하지 않다.

skip 구조를 활용하여 이미지의 전반적인 정보와 세부 정보를 혼합시킨다.

Fully convolutional networks

Adapting classifier for dense prediction



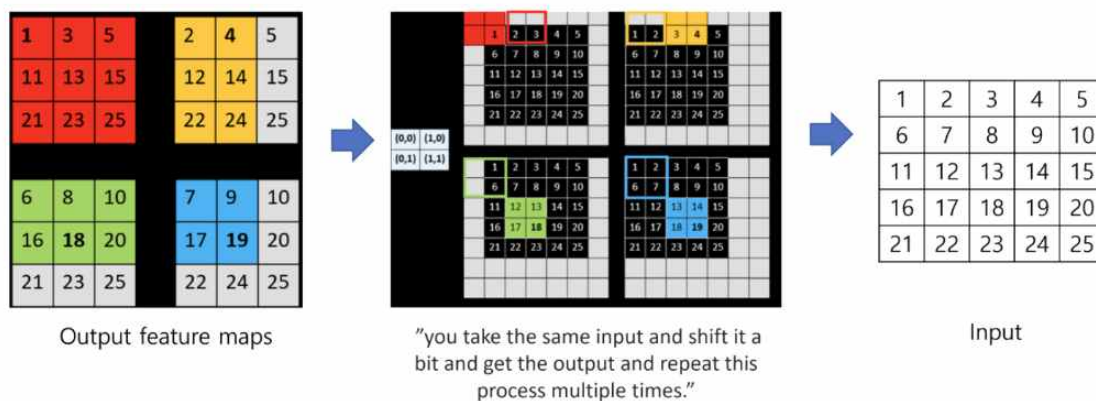
기존의 네트워크(LeNet, AlexNet 등)들은 아키텍처의 끝 부분에 FC layer가 있어 공간 정보

를 담지 못한채 output이 나오게 된다.

Segmentation의 경우 공간 정보를 유지하여 픽셀 단위로 예측해야 하므로 이러한 문제를 1x1 Conv를 활용하여 spatial dimension이 나오도록 만든다.

또한, 기존 CNN의 경우 input으로 패치를 받는 반면 FCN에서는 전체 이미지를 받아 효율적인 계산량을 가지게 된다.

Shift-and-stitch is filter rarefaction



coarse한 output으로부터 dense prediction을 얻는 방법으로 OverFeat 논문에서 쓰였던 Shift-and-stitch 방법 소개

Pooling 커널의 사이즈나 stride에 변화를 주는 경우 trade-off가 존재한다. Downsampling을 작게 할 경우 receptive field가 작아 연산량은 많으나 finer한 정보를 얻게 된다.

Shift-and-stitch trick을 쓰면 커널의 사이즈나 stride의 감소 없이도 패딩 방식을 다양하게 하여 dense prediction을 만들 수 있지만, receptive field가 작을수록 finer한 정보를 얻기에 제한적일 수 있다.

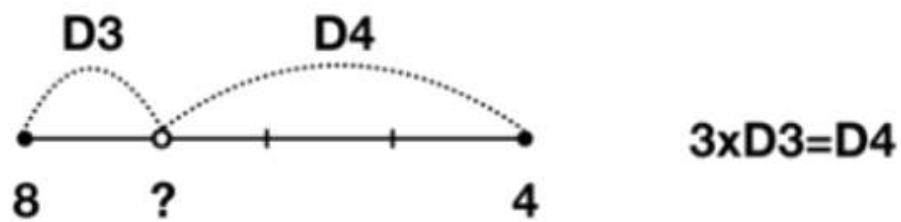
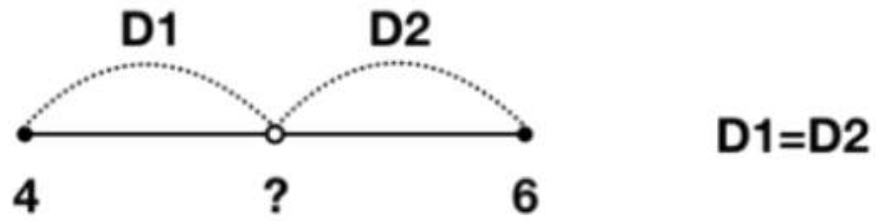
본 논문에서는 결과적으로 이 방식 대신 skip layer fusion을 사용한다.

Upsampling is backwards strided convolution

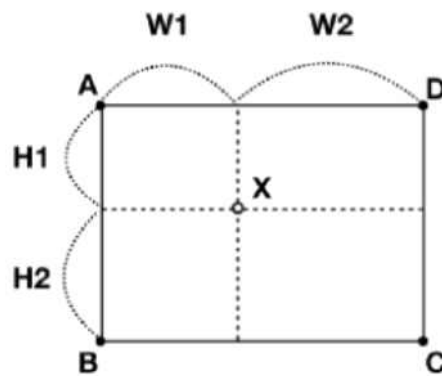
Upsampling을 하여 dense map을 얻는 방법으로 2가지 방법이 더 소개된다.

1)Interpolation

주어진 값들 사이의 값을 linear하게 추정한다.



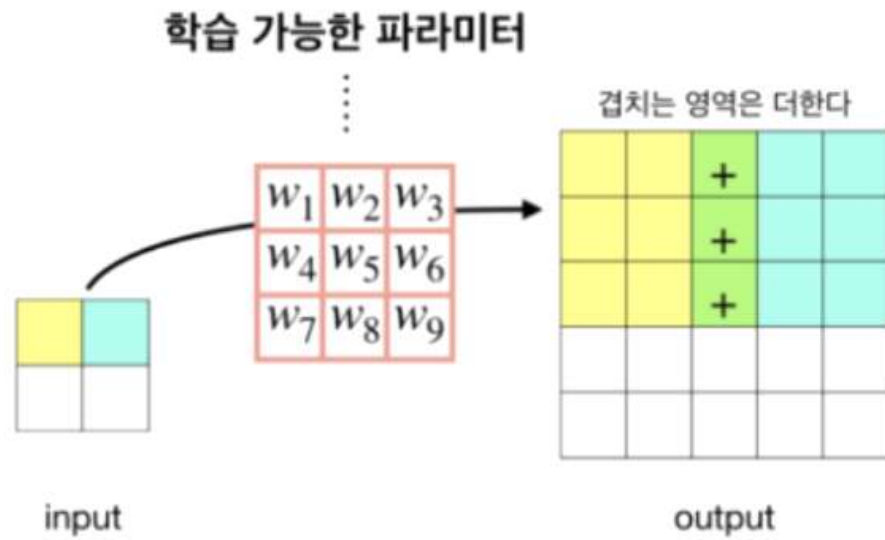
1D의 경우부터 살펴보면 위의 값은 5, 아래의 값은 7이라는 것을 쉽게 알 수 있다.



$$X = \left(A \frac{H2}{H1 + H2} + B \frac{H1}{H1 + H2} \right) \frac{W2}{W1 + W2} + \left(D \frac{H2}{H1 + H2} + C \frac{H1}{H1 + H2} \right) \frac{W1}{W1 + W2}$$

이를 2D로 확장하면 x,y 좌표를 각각 계산하면 된다. 이와 같은 방법으로 저해상도의 이미지를 고해상도의 이미지로 확장할 때 비어있는 중간값들을 유추하여 채워줄 수 있다.

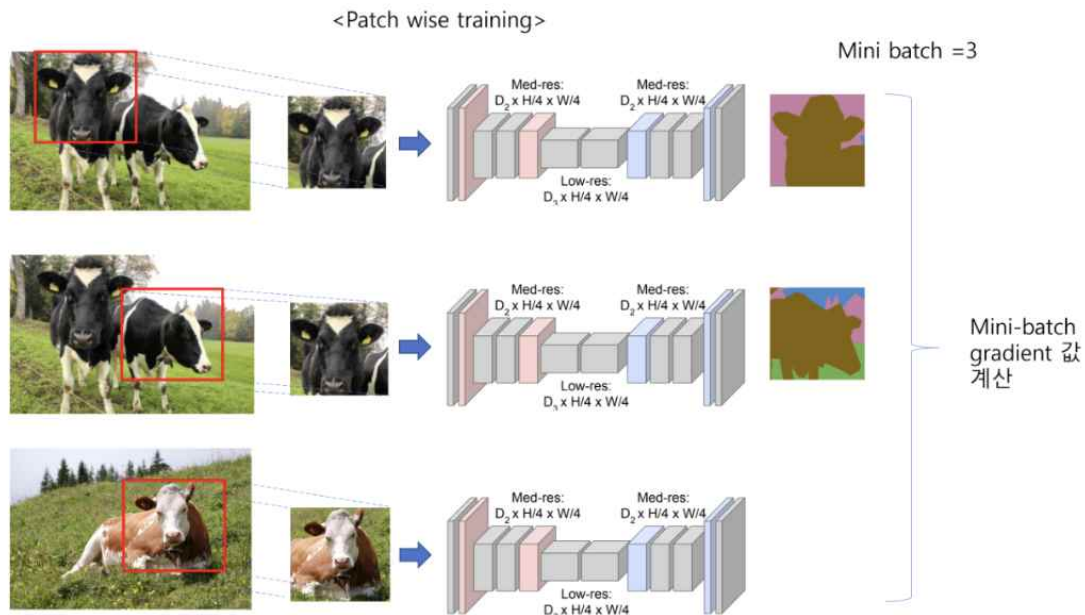
2)Deconvolution



Deconvolution은 Convolution의 역연산이다. 위의 그림과 같이 겹치는 영역은 더해서 구해 주면 Upsampling된 output을 얻을 수 있다.

하지만 2가지 방식 전부 이미 크게 줄어든 상태의 feature map으로부터 추정을 하는 형태기 때문에 여전히 정보손실은 클 수 밖에 없다. 따라서 본 논문에서는 skip architecture까지 적용해준다.

Patchwise training is loss sampling



patchwise 방식으로 학습할 경우 패치들 간에 겹치는 부분이 많아 불필요한 연산량이 많아질 수 있다. 반면, 전체 이미지를 사용할 경우 사이즈가 큰 경우 GPU 메모리를 많이 차지하여 충분한 미니배치를 잡기 어렵다.

패치 방식으로 학습을 시켜본 결과 더 빠르거나 loss가 잘 수렴하는 등의 장점은 없어 전체 이미지를 입력으로 받는 방식을 쓴다고 한다.

Segmentation Architecture

FCN은 Downsampling을 하는 Encoder 부분과 Upsampling을 하는 Decoder 부분으로 구성되어 있다.

Downsampling은 기존 CNN 구조와 동일하여 사전학습된 모델을 사용한다.

loss function은 각 픽셀마다 multinomial logistic loss가 사용되었다.

From classifier to dense FCN

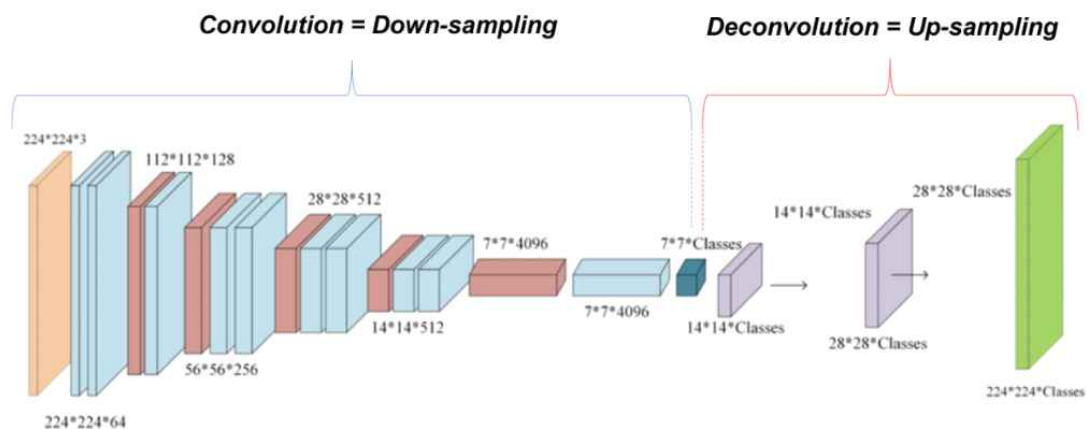
앞서 보았듯이 Classifier는 기존 모델들의 FC층을 Convolution으로 바꿔주어 사용한다.

AlexNet, VGG, GoogLeNet을 사용한 결과 VGG가 가장 좋은 성능을 보였다.

VGG와 GoogLeNet은 분류 task에서는 성능이 비슷함에도 불구하고 segmentation을 할 때는 GoogLeNet이 VGG를 따라오지 못했다.

Combining what and where

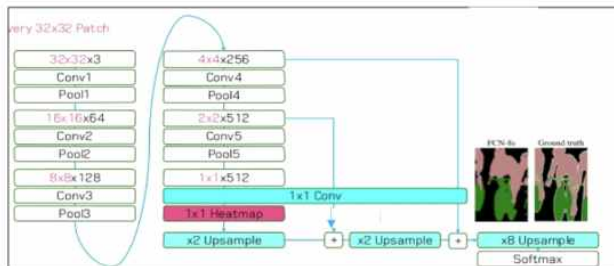
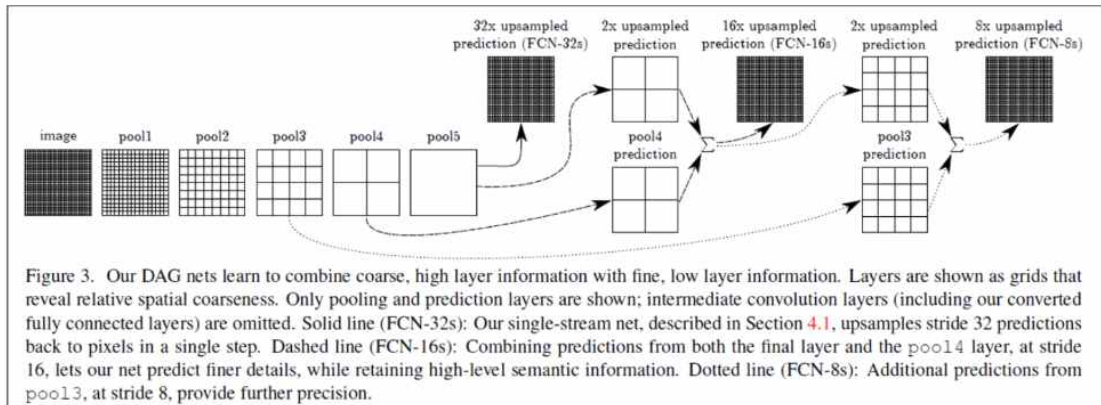
지금까지의 구조를 살펴보면 다음과 같고 해당 구조는 segmentation에서 충분한 성능을 보이지 못해 skip architecture를 제시한다.



논문에 나온 FCN-32는 32×32 의 이미지를 input으로 하고 1×1 으로 feature map을 뽑은 후 다시 32×32 로 upsampling한 결과를 의미한다.

이와 같이 Pooling 과정을 많이 거치는 경우 정보손실이 크기 때문에 upsampling을 하여 이미지의 사이즈를 맞춰준다 하더라도 fine information이 부족하여 segmeatation에서 좋은 성능을 보이기 어렵다.

따라서 pooling을 덜 거친 중간 단계의 feature map들을 활용하자는 것이 skip architecture의 핵심이다.



이미지의 좌측 하단 그림을 통해 FCN-16s를 설명하면 다음과 같다.

1. Pool5를 통해 생성된 1x1 feature map을 2배로 upsampling한다.
2. Pool4에서 생성된 2x2x512 feature map에서 1x1x512 필터를 적용한다.
3. 위의 두 과정에서 연산된 결과를 element-wise로 더해준 후, 16배로 upsampling하여 32x32로 만들어준다.